Databases 1

dr hab. inż. Przemysław Korytkowski, prof. ZUT

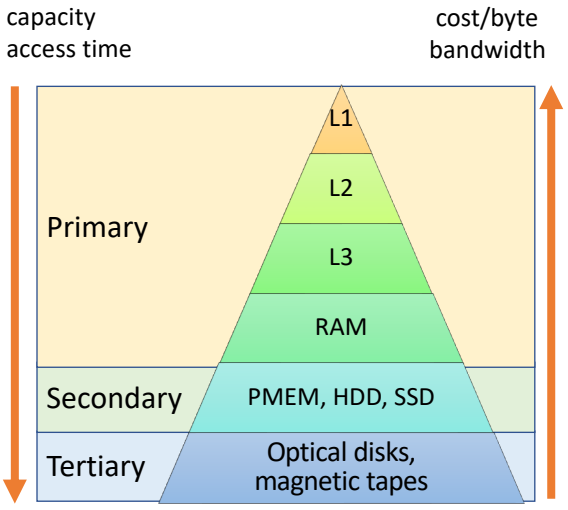
Lecture 5 – Storage Architectures

1

Storage hierarchy

AMD Ryzen 9 5950X
Cores: 16
L1 Cache Per Core: 64 KB
L2 Cache Per Core: 512 KB
L3 Cache Shared: 64 MB

Intel Core i9-12900K
Cores: 16 (8 P-Cores and 8 E-Cores)
L1 Cache Per P-Core: 80 KB
L2 Cache Per P-Core: 1280 KB
L1 Cache Per E-Core: 96 KB
L2 Cache Shared by E-Cores: 2 × 2 MB
L3 Cache Shared: 30 MB



2

Storage hierarchy

Primary storage	Secondary Storage	Tertiary storage
RAM, cache (L1, L2, L3)	**PMEM, HDD, SSD**	**Optical disks (CD-ROMs, DVDs), magnetic tapes**
Operated on directly by CPU	Data must be copied into primary storage and then processed by CPU	Data must be copied into primary storage and then processed by CPU
The most expensive		The least expensive
The highest data transfer		The lowers data transfer
The lowest capacity		The highest capacity

3

Storage hierarchy

- Generally, databases are too large to fit entirely in the main memory.
- The circumstances that cause permanent loss of stored data arise less frequently for disk secondary storage than for primary storage.
- Disk and other secondary storage devices are called **nonvolatile storage**,
- The main memory is called **volatile storage**.
- The cost of storage per unit of data is an order of magnitude less for disk secondary storage than for primary storage.
- Magnetic tapes are used for backing up databases because storage on tape costs much less than storage on disk.

4

Disk technologies performance comparison

persistent memory

Metric	HDD	SATA SSD	NVMe SSD	PMEM	Cloud Volume	RAM
IOPS (Random Reads)	~100	~10k	~500k	1M-3M	5k–50k	10M-20M
Latency (µs)	5k–10k	100–200	20–50	5–15	500–2k	0.03
Throughput (MB/s)	100–200	500–600	3k–7k	~10k	Variable	35k
Durability	High (mechanical)	High	High	Very high	Depends on replication	Volatile
Cost per GB	$	$$	$$$	$$$$	$$–$$$	$$$$

5

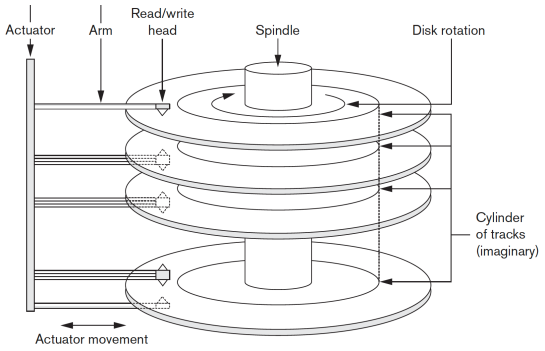
Disk technologies for RDBMS

Component	Optimal Storage	Reason
Data files	NVMe SSD	Fast random reads/writes for pages
Redo/transaction log	NVMe or PMEM	Lowest possible latency on fsync
Temporary tablespace	SSD (optional separate volume)	Reduces contention for temp operations
Archive logs / backups	HDD or cloud storage	Sequential writes, low cost
WAL replication logs (PostgreSQL)	SSD or NVMe	Low-latency streaming to replicas

6

HDD – Hard Disk Drive

- A disk is a random-access addressable device.
- Transfer of data between main memory and disk takes place in units of disk blocks.
- Search time:
  - **seek time** – mechanically position the
  - read/write head on the correct track, 4-10 ms
  - **rotational delay** for 15,000 rpm is 2 ms
  - **block transfer time**

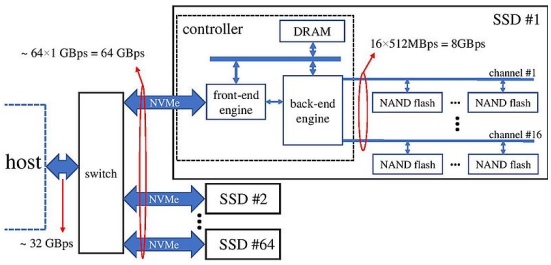


7

SSD – Solid Disk Drive

In comparison with HDD are :

- more resistant to physical shock,
- run silently,
- have higher IOPS
- lower latency
- more expensive
- have no mechanical parts
- Seek time is 0.08 – 0.16 ms



Source: Mahdi Torabzadehkashi, Siavash Rezaei, Ali HeydariGorji, Hosein Bobarshad, Vladimir Alves, and Nader Bagherzadeh, Public domain, via Wikimedia Commons

8

Random vs. Sequential access

Search time = seek time (s) + rotational delay (r) + block transfer time (t)

HDD – get 3 block of data

- Sequential:
  (s+r+t) + t + t
- Random:
  (s+r+t) + (s+r+t) + (s+r+t)

SSD – get 3 block of data

- Sequential:
  (s+t) + t + t
- Random:
  (s+t) + (s+t) + (s+t)

Random access is about slower than sequential x100 for HDD and x5 for SSD

9

Dominance of I/O cost

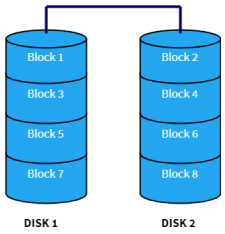
The time taken to perform a disk access is much larger than the time likely to be used manipulating that data in main memory.

Thus, the **number of block accesses** (Disk I /O 's) is a good approximation to the time needed by the algorithm and should be minimized.

10

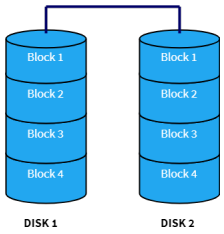
RAID - redundant arrays of inexpensive disks

RAID 0



- Offers great performance, both in read and write operations. There is no overhead caused by parity controls.
- Is not fault-tolerant. If one drive fails, all data in the RAID 0 array are lost.

RAID 1

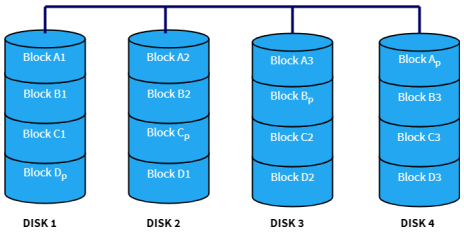


- Offers good read speed and a write-speed.
- In case a drive fails, data do not have to be rebuild, they just have to be copied to the replacement drive.
- only half of the total drive capacity because all data get written twice

11

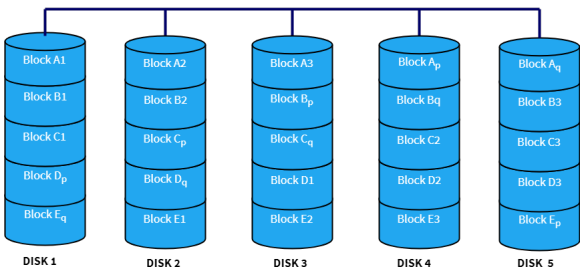
RAID - redundant arrays of inexpensive disks

RAID 5



- Read data transactions are very fast while write data transactions are slower (due to the parity that has to be calculated).
- If a drive fails, you still have access to all data.
- Drive failures have an effect on throughput

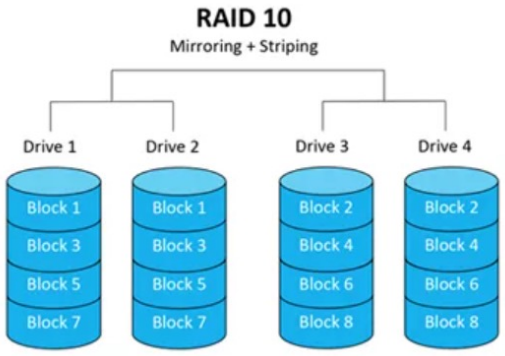
RAID 6



- Read data transactions are very fast.
- If two drives fail, you still have access to all data.
- Write data transactions are slower than RAID 5 due to the additional parity data that have to be calculated.

12

RAID - redundant arrays of inexpensive disks



- Use RAID 10 for high IO databases when possible, but keep in mind that 50% of the total disk space is used for mirroring.
- RAID 5 / RAID 6 have slower write performance, but fast read performance, and are suitable for read-only databases.
- Logs should be written to separate RAID arrays from data files.

Component	OLTP	OLAP
System / OS	RAID 1	RAID 1
Database Data	RAID 10	RAID 5 or 6
Transaction Log / WAL	RAID 1 or 10	RAID 1
Temp Space	RAID 0 (or 10)	RAID 0
Backups / Archives	RAID 5/6	RAID 6

13

Hardware RAID

- Cost is higher
- Better performance
- More configurations (5,6 10 not supported by software RAID)
- Two options:
  - Bus-based (come with the motherboard)
  - Card-based and intelligent controllers (high-end systems with dedicated processors)

Software RAID

- Cost effective
- Slower
- Supports more drives
- Problematic disc replacement

14

Disk Management options

- **Direct Attached Storage (DAS)**

  DAS is a fast, easy to use option for SQL storage. However, DAS can be limited in terms of capacity and available RAID configuration.
- **Network Attached Storage (NAS)**

  NAS devices usually contain one or more RAID arrays which are accessed remotely over TCP/IP. Rather than accessing data at a block level, data on a NAS is accessed at the file level – **unsuitable for databases**.
- **Storage Area Network (SAN)**

  SAN devices can be configured to contain one or more RAID arrays and are usually attached through a Fibre Channel, Fibre Channel over Ethernet, or iSCSI connection. SAN appears to a database as if it were DAS, allowing block level access.

15

File organizations

Primary file organizations determines how the file records are physically placed on the disk:

- **A heap file** (or unordered file) places the records on disk in no particular order by appending new records at the end of the file (MySQL, Oracle, SQL Server)
  - PostgreSQL with Free Space Map
- **A sorted file** (or sequential file) data is stored in order of the clustering key (usually the primary key). The index and data are stored together.
- **A hashed file** uses a hash function applied to a particular field (called the hash key) to determine a record's placement on disk.
- **B-trees**, use tree structures.

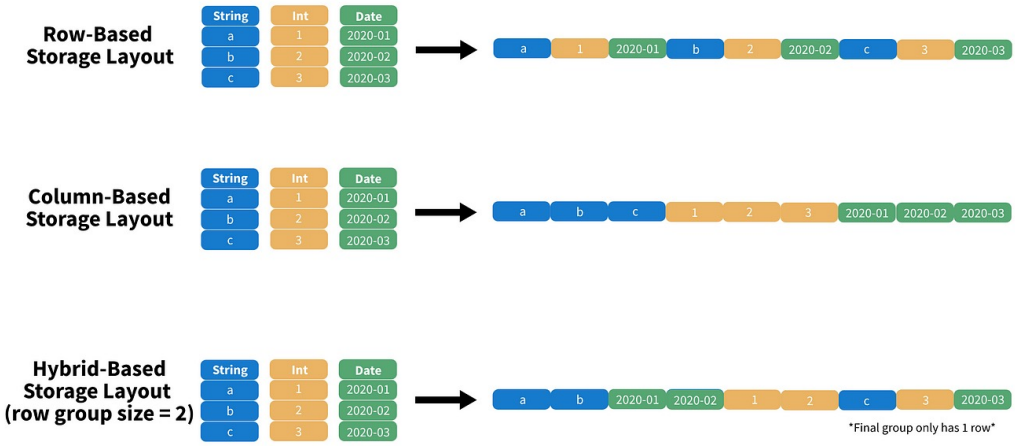
16

File organizations

Technique	Read Performance	Insert Performance	Range Query	Maintenance	Typical Use
Heap	Medium	Excellent	Poor	Low	OLTP inserts
Sorted	Excellent	Moderate	Excellent	High	OLAP / PK access
Hashed	Excellent (equality)	Good	Poor	Moderate	Key-value lookups
B-Tree	Excellent	Moderate	Good	High	Multi-attribute search

17

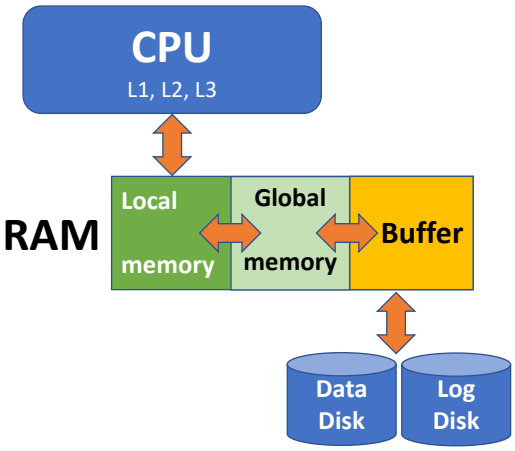
Row and Column oriented file storage



18

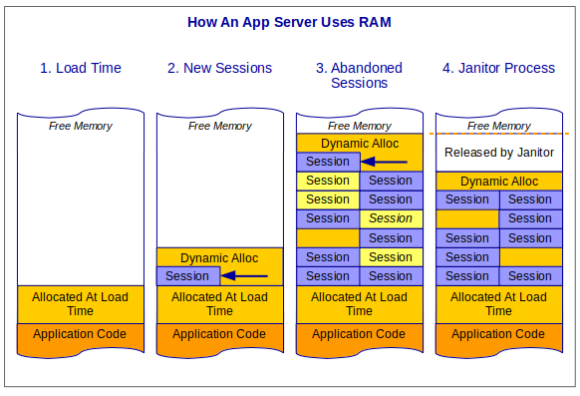
Regions of memory

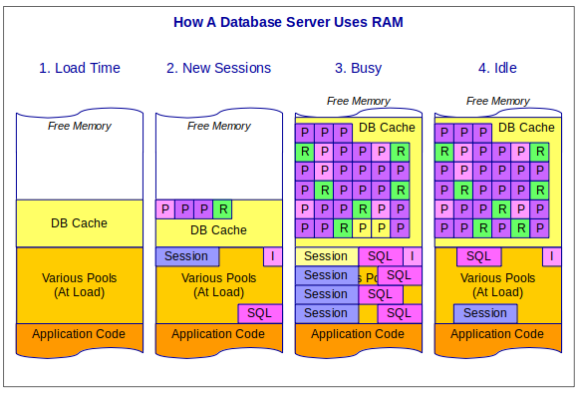
- Typical database applications need only a small portion of the database at a time for processing.
- Whenever a certain portion of the data is needed, it must be located on disk, copied to main memory for processing, and then rewritten to the disk if the data is changed.
- The data stored on disk is organized as files of records. Each record is a collection of data values that can be interpreted as facts about entities, their attributes, and their relationships.
- Records should be stored on disk in a manner that makes it possible to locate them efficiently when they are needed.



19

Memory allocation

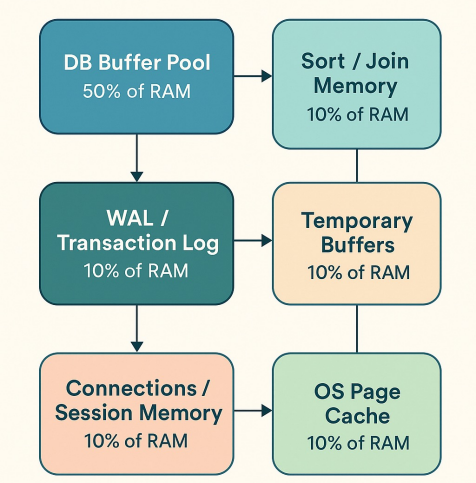


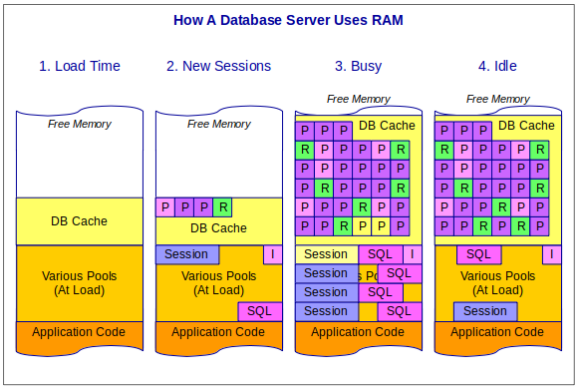


Source: https://justinparrtech.com/JustinParr-Tech/how-database-servers-use-memory/

20

Memory management in MariaDB





21

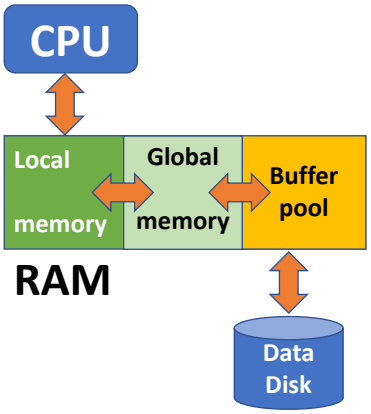
Buffer manager

- To reduce the number of accesses to persistent storage, pages are cached in memory. When the page is requested again by the storage layer, its cached copy is returned.
- **Virtual disk** or **page cache** or **buffer pool** – an approach that assumes that cached pages available in memory can be reused and no other process can modify the data on disk.
- A virtual disk read accesses physical storage only if no copy of the page is already available in memory.
- In case of a database system crash or unorderly shutdown, cached contents are lost.

22

Buffer manager functions

- It keeps cached page contents in memory.
- It allows modifications to on-disk pages to be buffered together and performed against their cached versions.
- When a requested page isn't present in memory and there's enough space available for it, it is **paged in** by the page cache, and its cached version is returned.
- If an already cached page is requested, its cached version is returned.
- If there's not enough space available for the new page, some other page is **evicted** and its contents are **flushed** to disk.



23

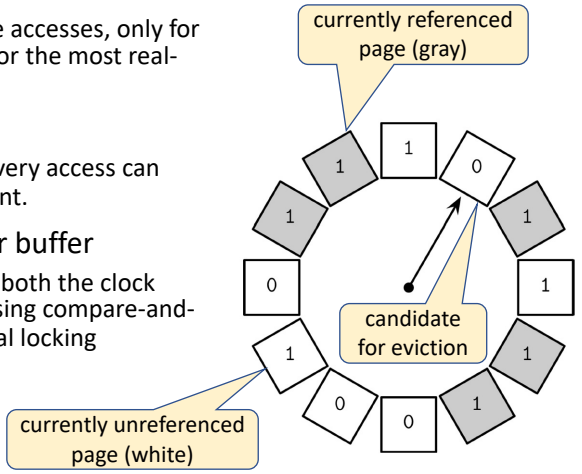
Buffer manager

- The goal of the **buffer manager are**:
  - to maximize the probability that the requested page is found in main memory,
  - in case of reading a new disk block from disk, to find a page to replace that will cause the least harm in the sense that it will not be required shortly again.
- The buffer manager keeps two types of information about each page in the buffer pool:
  - A **pin-count**: the number of times that page has been requested, or the number of current users of that page. If this count falls to zero, the page is considered unpinned. Initially the pin-count for every page is set to zero. Incrementing the pin-count is called pinning. In general, a pinned block should not be allowed to be written to disk.
  - A **dirty bit**, which is initially set to zero for all pages but is set to 1 whenever that page is updated by any application program.

24

Page Replacement (eviction) policies

- first in first out (FIFO)
  - Since it does not account for subsequent page accesses, only for page-in events, this proves to be impractical for the most real-world systems.
- least recently used (LRU)
  - updating references and relinking nodes on every access can become expensive in a concurrent environment.
- clock replacement – round-robin – circular buffer
  - An advantage of using a circular buffer is that both the clock hand pointer and contents can be modified using compare-and-swap operations, and do not require additional locking mechanisms.



25

Buffers – prefetching

- A DBMS can often predict reference patterns because most page references are generated by higher-level operations (such as sequential scans or implementations of various relational algebra operators) with a known pattern of page accesses.
- This allows for a better choice of pages to replace and makes the idea of specialized buffer replacement policies more attractive in the DBMS environment than by operational system.
- **Prefetching**: the buffer manager can anticipate the next several page requests and fetch the corresponding pages into memory before the pages are requested and thus:
  - the pages are available in the buffer pool when they are requested
  - reading in a contiguous block of pages is much faster than one by one

26

Buffer – Lock pages

- **Lock pages** (pinned) are kept in memory for a longer time, which helps to reduce the number of disk accesses and improve performance.
- We can lock pages that have a high probability of being used in the nearest time:
  - Top levels of B^+ trees indexes,
  - hash indexes

27

Thank you for your attention!

Time for questions

28

28